

Problem Set 1: Python Fundamentals

Prof. Jordan C. Hanson

August 24, 2026

1 Exercise 1: Evaluating a Function

1. The Python interpreter can be used as a calculator. In this exercise, use the interpreter to evaluate the cosine function at ten different values of x .

- Begin by importing the cosine function from the Python `math` library:

```
from math import cos
```

- The function `cos(x)` expects x to be measured in radians. Evaluate the cosine at the following ten values:

$x = 0.0, 0.5, 1.0, 1.5, 2.0,$
 $2.5, 3.0, 3.5, 4.0, 4.5.$

- For each value, use the interpreter to print both x and `cos(x)`. For example:

```
print(0.0, cos(0.0))  
print(0.5, cos(0.5))
```

- Continue until all ten points have been evaluated. Your output should contain two columns: the value of x followed by the value of `cos(x)`.
- Copy the ten lines of numerical output from the interpreter and paste them into a spreadsheet program. Create an x - y plot with x on the horizontal axis and `cos(x)` on the vertical axis.
- Does the shape of your graph resemble the cosine function that you expect? Identify approximately where the function first crosses zero.

2 Exercise 2: Report Card

1. Write a Python program called `report_card.py`. The program will collect a student's final grades using the `input()` function and then print a formatted report card.

The courses are:

Computer Science 1
Calculus I
General Physics I
English Composition
First-Year Seminar

- Begin by asking the user to enter the student's name:

```
name = input("Student name: ")
```

- Next, use one `input()` statement for each course to collect the student's final letter grade. For example:

```
cs_grade = input(  
    "Computer Science 1 grade: ")
```

- Store each response in a variable. Your program should therefore contain one variable for the student's name and one variable for the grade in each of the five courses.

- After all of the information has been entered, print the report card. The course names should appear on the left and the final grades should appear on the right, separated by periods. For example, the final output should have the form:

```
Report Card  
Student: Ada Lovelace  
  
Computer Science 1.....A  
Calculus I.....B+  
General Physics I.....A-  
English Composition.....B  
First-Year Seminar.....A
```

- The grades shown above are only an example. The grades appearing in the final report card must be the values entered by the user through the `input()` prompts.

- Use the same alignment technique from the table-of-contents warm-up. Add enough periods so that the grades appear in approximately the same column.

- Run the program at least twice using two different sets of grades. Verify that the final report card changes when different information is entered.

3 Exercise 3: Class Rosters

1. Write a Python program called `class_rosters.py`. The program will organize the rosters of three courses. Each course contains exactly three students.

Use the following courses and students:

Computer Science 1

Ada Lovelace
Alan Turing
Grace Hopper

Calculus I

Katherine Johnson
Isaac Newton
Emmy Noether

General Physics I

Marie Curie
Albert Einstein
Richard Feynman

- Represent the three student names for each course using a Python list. For example:

```
students = [
    "Ada Lovelace",
    "Alan Turing",
    "Grace Hopper"
]
```

- Each individual class roster should also be a list containing exactly two elements. The first element should be the name of the course, and the second element should be the list containing the three student names.

The structure of one roster should therefore have the form:

```
[
    "Computer Science 1",
    ["Ada Lovelace",
     "Alan Turing",
     "Grace Hopper"]
]
```

- Construct one roster for each of the three courses.
- Finally, combine the three class rosters into one tuple. The final object must therefore have the structure

```
(
    [course1, [student1, student2,
               student3]],
    [course2, [student1, student2,
               student3]],
    [course3, [student1, student2,
               student3]]
)
```

- The outer object must be a tuple containing three elements. Each of those elements must be a list containing a course name and a second list of three student names.
- Assign the final tuple to a variable called `class_rosters` and use `print(class_rosters)` to inspect the completed data structure.
- Sketch the nested structure of `class_rosters` on paper before writing the complete Python statement. Clearly identify which brackets represent lists and which parentheses represent the tuple.